

ЗАЩИТА БАЗЫ: МЕТЕОРИТНЫЙ ДОЖДЬ

МОДУЛЬ: СОБЫТИЙНАЯ АНИМАЦИЯ, КООРДИНАТЫ И СЛУЧАЙНЫЕ СОБЫТИЯ | ID: 6

ЭТАП 1: 1. Модель данных для системы повреждений (Пишем вне экрана)

```
// Класс для хранения параметров текущей угрозы.
// Пишем в самом верху файла, вне функций.
data class Meteorite(
    val xOffset: Int, // Случайная позиция по горизонтали
    val yOffset: Int, // Случайная позиция по вертикали
    val size: Int,     // Размер цели
    val reward: Int    // Очки за попадание
)
```

ЭТАП 2: 2. Создаем элемент UI: Индикатор прочности щита (ShieldBar)

```
// Отдельный UI-элемент для отображения здоровья базы.
// Вынесение его из главного экрана упрощает чтение кода.
@Composable
fun ShieldBar(hp: Int) {
    Column(modifier = Modifier.fillMaxWidth().padding(8.dp)) {
        Text("ПРОЧНОСТЬ ЭНЕРГОЩИТА: $hp%", color = Color.White, fontSize = 12.sp)
        Spacer(modifier = Modifier.height(4.dp))
        LinearProgressIndicator(
            progress = { hp / 100f },
            modifier = Modifier.fillMaxWidth().height(8.dp).clip(RoundedCornerShape(4.dp)),
            color = if (hp < 30) Color.Red else Color(0xFF00FFCC)
        )
    }
}
```

ЭТАП 3: 3. Объявление главного экрана игры

```
// Заготовка для главного экрана.
// Код из следующих шагов пишем строго внутри этой функции.
@Composable
fun MeteoriteDefenseScreen() {
    // В самом конце файла не забудь поставить закрывающую скобку '}'!
```

ЗАЩИТА БАЗЫ: МЕТЕОРИТНЫЙ ДОЖДЬ

МОДУЛЬ: СОБЫТИЙНАЯ АНИМАЦИЯ, КООРДИНАТЫ И СЛУЧАЙНЫЕ СОБЫТИЯ | ID: 6

ЭТАП 4: 4. Переменные состояния игрового процесса

```
// Переменные для отслеживания очков, здоровья щита и состояния метеорита.
var score by remember { mutableStateOf(0) }
var shieldHp by remember { mutableStateOf(100) }
var activeMeteorite by remember { mutableStateOf<Meteorite?>(null) }

// Время (в миллисекундах), оставшееся на то, чтобы успеть нажать на метеорит
var timeLeft by remember { mutableStateOf(0L) }
val maxTimeForReaction = 1500L // 1.5 секунды на клик
```

ЭТАП 5: 5. Жизненный цикл игры: Спавн метеоритов (LaunchedEffect 1)

```
// Этот цикл постоянно работает в фоне и каждые 1-2 секунды создает метеорит.
LaunchedEffect(shieldHp) {
    if (shieldHp > 0) {
        while (shieldHp > 0) {
            // Ждем случайное время перед атакой метеорита (от 1 до 2.5 секунд)
            delay(kotlin.random.Random.nextLong(1000, 2500))

            if (activeMeteorite == null) {
                // Генерируем метеорит в случайных координатах на экране
                activeMeteorite = Meteorite(
                    xOffset = kotlin.random.Random.nextInt(-120, 120),
                    yOffset = kotlin.random.Random.nextInt(-200, 100),
                    size = kotlin.random.Random.nextInt(50, 80),
                    reward = 100
                )
                timeLeft = maxTimeForReaction
            }
        }
    }
}
```

ЗАЩИТА БАЗЫ: МЕТЕОРИТНЫЙ ДОЖДЬ

МОДУЛЬ: СОБЫТИЙНАЯ АНИМАЦИЯ, КООРДИНАТЫ И СЛУЧАЙНЫЕ СОБЫТИЯ | ID: 6

ЭТАП 6: 6. Жизненный цикл игры: Таймер реакции (LaunchedEffect 2)

```
// Этот эффект следит за временем жизни метеорита.
// Запускается и работает только тогда, когда на экране есть активная угроза.
LaunchedEffect(activeMeteorite) {
    if (activeMeteorite != null) {
        while (timeLeft > 0) {
            delay(50) // Обновляем таймер каждые 50 миллисекунд
            timeLeft -= 50
        }

        // Если время вышло, а метеорит не уничтожен щит получает урон
        if (activeMeteorite != null) {
            shieldHp = (shieldHp - 20).coerceAtLeast(0)
            activeMeteorite = null // Метеорит исчезает
        }
    }
}
```

ЭТАП 7: 7. Основной контейнер экрана (Box)

```
// Используем Box, так как нам нужно накладывать элементы друг на друга
// (метеорит должен свободно летать поверх фона).
Box(
    modifier = Modifier.fillMaxSize().background(Color(0xFF0B0E14)),
    contentAlignment = Alignment.Center
) {
    // Фоновая сетка или элементы интерфейса
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        verticalArrangement = Arrangement.SpaceBetween,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        // Верхняя панель счета
        Text("СБИТО МЕТЕОРИТОВ: $score", color = Color(0xFFFFD54F), fontSize = 20.sp,
            fontWeight = FontWeight.Bold)

        // Нижняя панель (перенесем в шаг 8)
    }
}
```

ЗАЩИТА БАЗЫ: МЕТЕОРИТНЫЙ ДОЖДЬ

МОДУЛЬ: СОБЫТИЙНАЯ АНИМАЦИЯ, КООРДИНАТЫ И СЛУЧАЙНЫЕ СОБЫТИЯ | ID: 6

ЭТАП 8: 8. Панель управления и статусная зона

```
// Вставляем этот блок в самый низ Column из Шага 7 (вместо комментария)
Column(horizontalAlignment = Alignment.CenterHorizontally) {
    ShieldBar(hp = shieldHp)

    Spacer(modifier = Modifier.height(12.dp))

    if (shieldHp <= 0) {
        Button(
            onClick = {
                shieldHp = 100
                score = 0
                activeMeteorite = null
            },
            colors = ButtonDefaults.buttonColors(containerColor = Color.Red)
        ) {
            Text("НАЧАТЬ ЗАНОВО", color = Color.White)
        }
    }
}
```

ЗАЩИТА БАЗЫ: МЕТЕОРИТНЫЙ ДОЖДЬ

МОДУЛЬ: СОБЫТИЙНАЯ АНИМАЦИЯ, КООРДИНАТЫ И СЛУЧАЙНЫЕ СОБЫТИЯ | ID: 6

ЭТАП 9: 9. Отрисовка активной угрозы (Метеорита)

```
// Код пишется внутри Box (Шаг 7), на одном уровне с Column.
// Метеорит отображается только если игра активна и объект создан.
activeMeteorite?.let { met ->
    Box(
        modifier = Modifier
            .offset(x = met.xOffset.dp, y = met.yOffset.dp) // Случайная позиция
            .size(met.size.dp)
            .clip(CircleShape)
            .background(Color(0xFFE57373))
            .clickable {
                // Успешное попадание!
                score += 1
                activeMeteorite = null // Метеорит уничтожен
            },
        contentAlignment = Alignment.Center
    ) {
        // Показываем уменьшающийся индикатор времени внутри метеорита
        CircularProgressIndicator(
            progress = { timeLeft.toFloat() / maxTimeForReaction },
            color = Color.White,
            strokeWidth = 4.dp
        )
        Text("💣 ", fontSize = (met.size / 2).sp)
    }
}
```

ЭТАП 10: 10. Предпросмотр Экрана (Preview)

```
// Пишется СНАРУЖИ основной функции MeteoriteDefenseScreen.
@Preview
@Composable
fun MeteoriteDefensePreview() {
    MaterialTheme {
        MeteoriteDefenseScreen()
    }
}
```

ЗАЩИТА БАЗЫ: МЕТЕОРИТНЫЙ ДОЖДЬ

МОДУЛЬ: СОБЫТИЙНАЯ АНИМАЦИЯ, КООРДИНАТЫ И СЛУЧАЙНЫЕ СОБЫТИЯ | ID: 6

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь золотые метеориты: создай переменную `isGolden`. Сделай так, чтобы при появлении метеорита с шансом 15% (используй `Random.nextFloat() < 0.15f`) он становился золотым. Измени цвет кнопки золотого метеорита на `Yellow`, а награду за его уничтожение сделай +500 энергии вместо обычных +100.